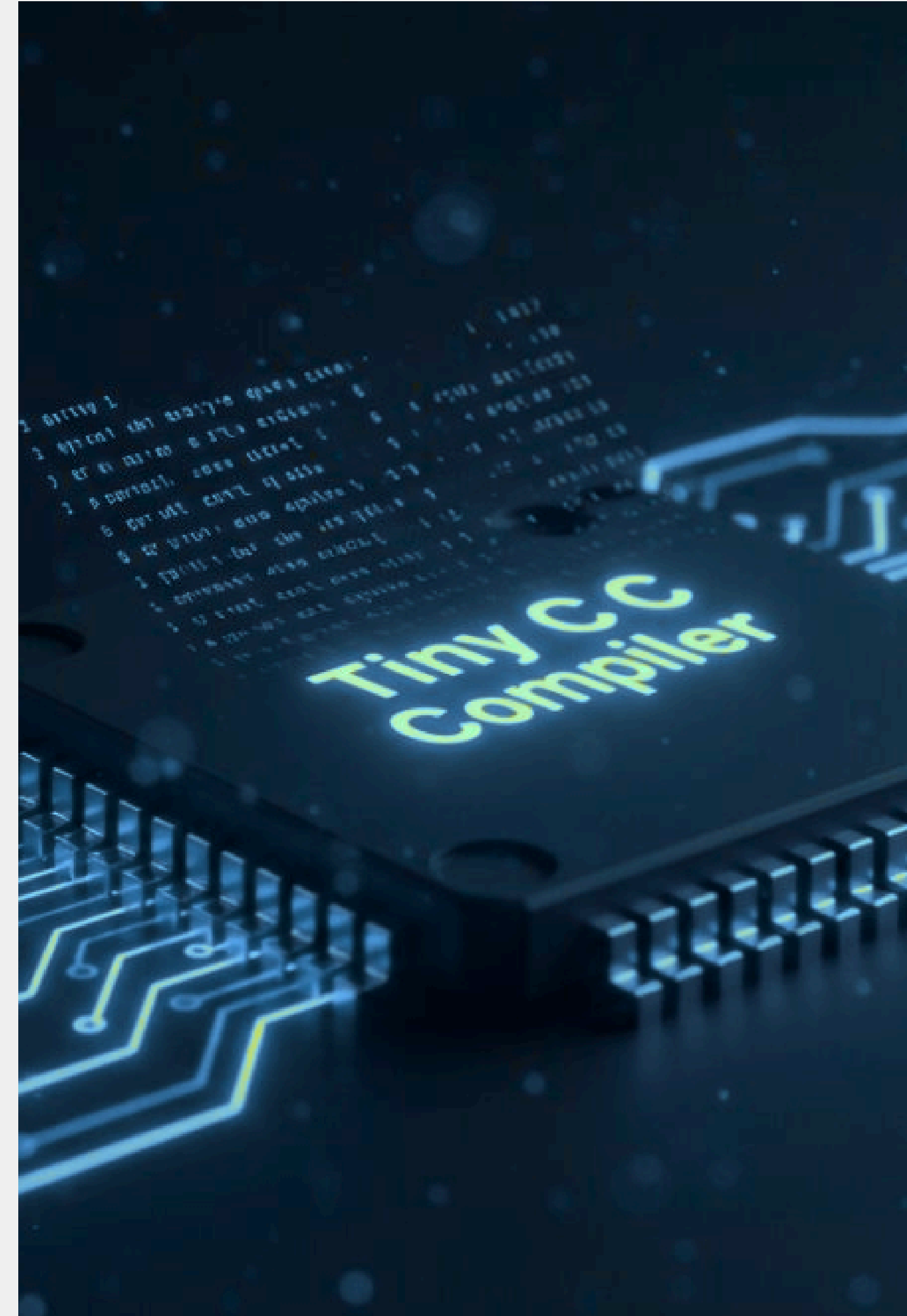




JULIA TIZUEY MARTINS HOSOY

# Projeto A3: Protótipo de Compilador (MiniCalc)

Teoria da Computação e Compiladores



# ○ Estudo de Caso

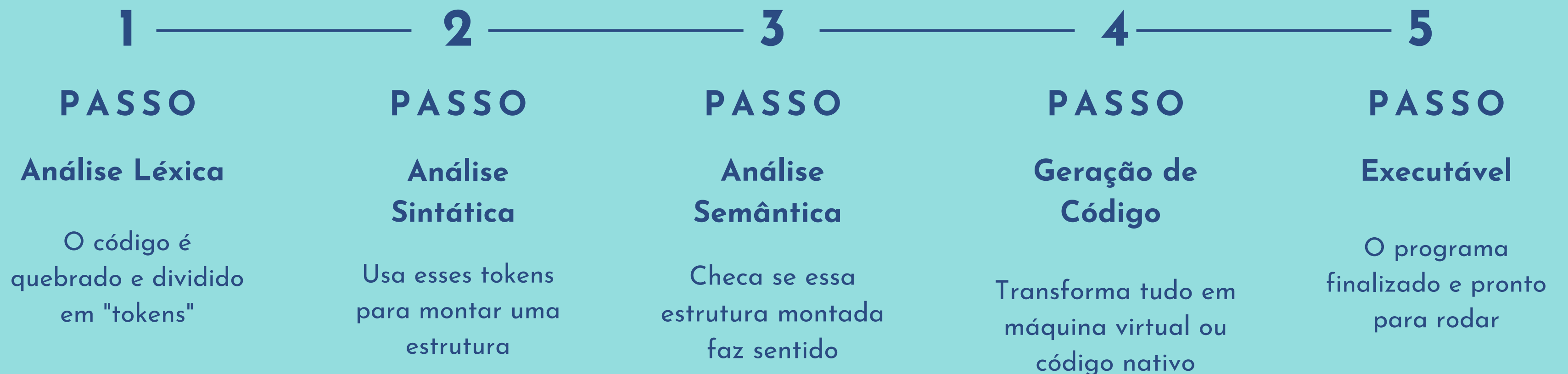
**Compilador estudado:** Tiny C Compiler (TCC).

- Compilador para a linguagem C.
- Pequeno, rápido e com código-fonte fácil de entender.
- Possui as principais etapas presentes em compiladores maiores.

**○ que foi aproveitado no projeto**

- O MiniCalc foi desenvolvido seguindo a mesma ideia de dividir o processo em etapas: análise léxica, análise sintática, construção da AST e geração de código.

# Fluxo simplificado do TCC





# Arquitetura do MiniCalc

- O MiniCalc recebe um arquivo .calc e gera um código Python equivalente.

## Fluxo do compilador:

- Código Fonte → Lexer → Parser → AST → Geração de Código Python
- Cada etapa possui uma responsabilidade específica, facilitando a organização do projeto e a identificação de erros.



# Front-end: Lexer e Parser

## Lexer

- O Lexer foi implementado utilizando Expressões Regulares da biblioteca re do Python.
- Ele é responsável por ler o código-fonte e identificar seus elementos, como números, operadores, identificadores e palavras-chave, transformando tudo em uma sequência de tokens.

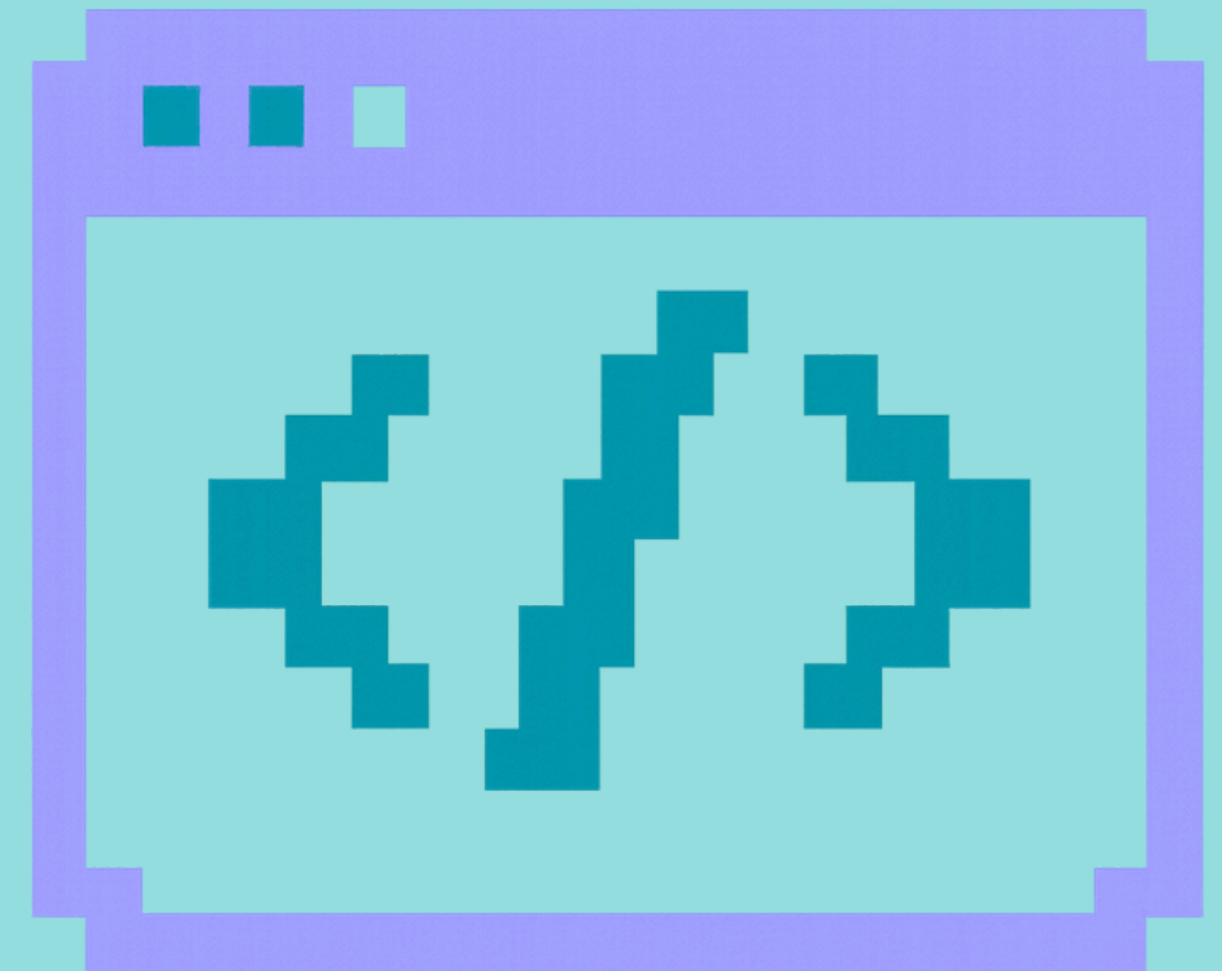
## Parser

- Cada regra da gramática possui um método correspondente, o que tornou o código mais organizado e fácil de relacionar com a teoria estudada em sala.
- Durante essa etapa também é construída a AST, que garante automaticamente a precedência correta das operações matemáticas.



# Back-end: Geração de Código

- Após a construção da AST, o compilador percorre cada nó da árvore para gerar o código Python equivalente.
- Por exemplo, um nó de atribuição é convertido em uma instrução de atribuição do Python, enquanto um nó de operação matemática é convertido em uma expressão com os operadores correspondentes.
- Como a árvore já foi validada pelo Parser, a geração do código acontece de forma direta, produzindo um código Python válido e pronto para ser executado.



# Demonstração Prática

## (Exemplo de Execução)

### CÓDIGO FONTE ORIGINAL DE ENTRADA (.CALC):

```
≡ ex3_print.calc U X
minicalc_TrilhaDeAprendizado > examples > ≡ ex3_print.calc
1  base = 5
2  altura = 10
3  area = base * altura / 2
4  print area
```

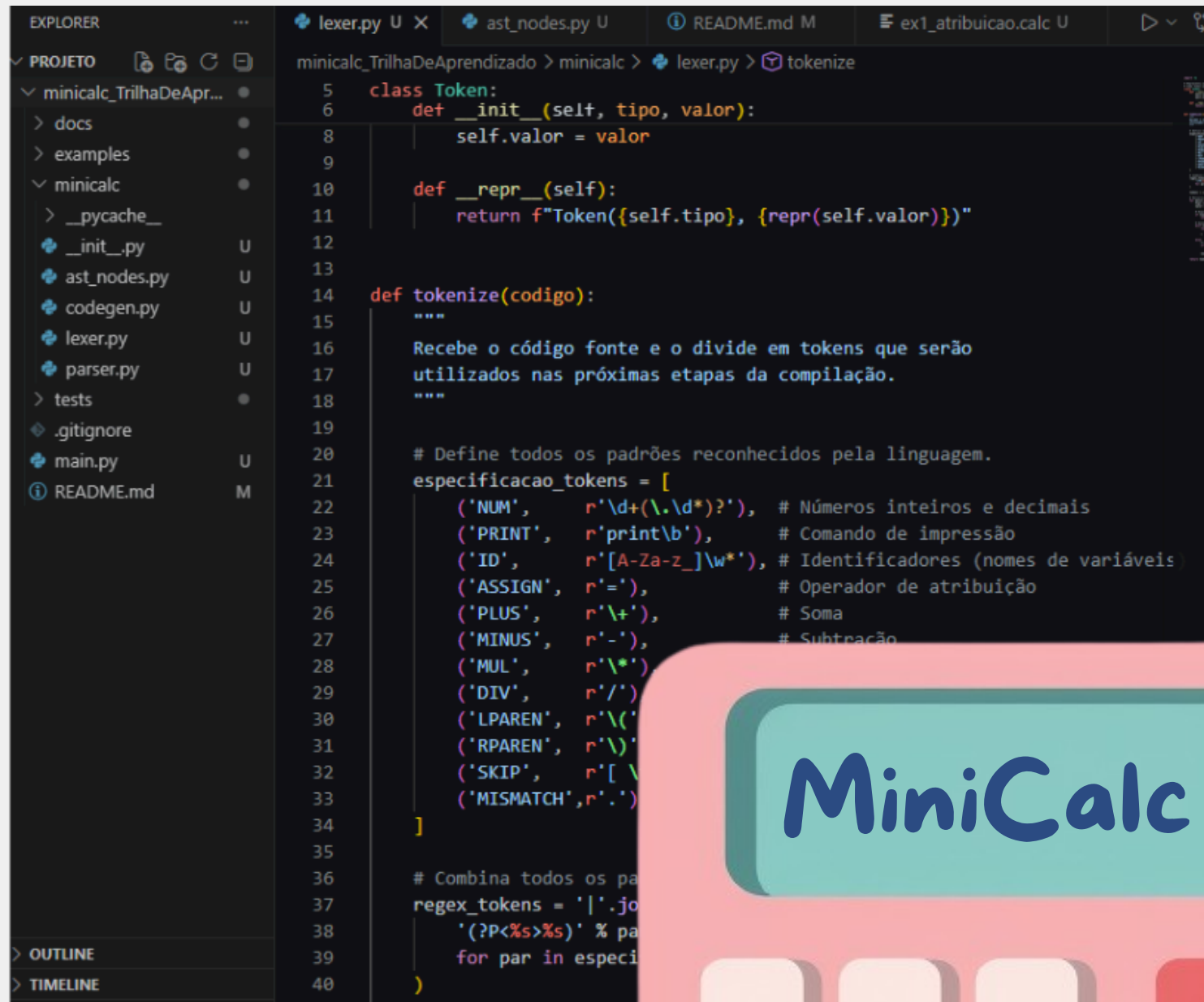
### CÓDIGO FINAL GERADO COM SUCESSO (PYTHON):

```
base = 5
altura = 10
area = ((base * altura) / 2)
print(area)
```

### PROCESSAMENTO INTERNO DO COMPILADOR:

- **Lexer:** Lê o texto e extrai os tokens em sequência: [ID: area], [ASSIGN: =], [ID: base], [MUL: \*], [ID: altura], [DIV: /], [NUM: 2]...
- **Parser:** Monta a Árvore Sintática agrupando a multiplicação e a divisão de forma hierárquica ((base \* altura) / 2).





```
5 class Token:
6     def __init__(self, tipo, valor):
7
8         self.valor = valor
9
10    def __repr__(self):
11        return f"Token({self.tipo}, {repr(self.valor)})"
12
13
14    def tokenize(codigo):
15        """
16        Recebe o código fonte e o divide em tokens que serão
17        utilizados nas próximas etapas da compilação.
18        """
19
20        # Define todos os padrões reconhecidos pela linguagem.
21        especificacao_tokens = [
22            ('NUM', r'\d+(\.\d*)?'), # Números inteiros e decimais
23            ('PRINT', r'print\b'), # Comando de impressão
24            ('ID', r'[A-Za-z_]\w*'), # Identificadores (nomes de variáveis)
25            ('ASSIGN', r'='), # Operador de atribuição
26            ('PLUS', r'\+'), # Soma
27            ('MINUS', r'-'), # Subtração
28            ('MUL', r'\*'),
29            ('DIV', r'/'),
30            ('LPAREN', r'\('),
31            ('RPAREN', r'\)'),
32            ('SKIP', r'[ \t\n]'),
33            ('MISMATCH', r'.'),
34        ]
35
36        # Combina todos os padrões em uma única expressão regular.
37        regex_tokens = '|'.join(f'({pattern})' for pattern in especificacao_tokens)
38        regex_tokens = f'({regex_tokens})'
39        for par in especificacao_tokens:
40            pass
```



# Conclusão Geral

- **Teoria na Prática:** A aplicação de autômatos e linguagens formais provou ser a fundação essencial e útil para a engenharia de software na prática.
- **Robustez:** O rigor sintático estabelecido no projeto previne que falhas lógicas drásticas cheguem ao usuário.
- **Escalabilidade:** O protótipo desenvolvido possui uma estrutura altamente escalável, permitindo a fácil inclusão de novas estruturas de controle (como if e while) em expansões futuras.



**Muito obrigada  
professora pela  
oportunidade.**

[https://github.com/Tizuey/minicalc\\_TrilhaDeAprendizado](https://github.com/Tizuey/minicalc_TrilhaDeAprendizado)

